

CS313 Deep Learning for Artificial Intelligence

SPRING 2026

Final Project Report

Time-Series Data and Application to Stock Markets

Nguyen Phuc Minh Anh

Student ID: 240141

GitHub Repository: <https://github.com/autumn0910/DL4AI-240141-project>

Contents

1	Introduction	3
2	Datasets and Preprocessing	3
2.1	Nasdaq Dataset	3
2.2	Vietnam Dataset	4
2.3	Preprocessing Pipeline	4
3	Task 1 — Nasdaq Stock Price Prediction	5
3.1	Model Architecture	5
3.2	Task 1.1 — Multi-Feature Extension	5
3.3	Task 1.2 — N-th Day Forecast	6
3.4	Task 1.3 — K Consecutive Days Forecast	6
4	Task 2 — Vietnam Stock Price Prediction	6
4.1	Data Specifics	6
4.2	Task 2.1 — Multi-Feature Extension	7
4.3	Task 2.2 — N-th Day Forecast	7
4.4	Task 2.3 — K Consecutive Days Forecast	7
5	Task 3 — Trading Signal Identification	7
5.1	Problem Formulation	7
5.2	Feature Engineering	8
5.3	Model Architecture: LSTM + Gradient Boosting Hybrid	8

5.4	Results	9
6	Task 4 — Portfolio Composition and Risk Management	9
6.1	Universe Selection	9
6.2	Task 4.1 — Profitable Stock Selection	9
6.3	Task 4.2 — Risk Management	10
6.4	Task 4.3 — Portfolio Composition	10
7	Task 5 — Deployment and AI Engineering (Extra Credit)	11
7.1	Task 5.1 — Model Deployment as API	11
7.2	Task 5.2 — Model as SaaS	11
7.3	Task 5.3 — AI Engineering Workflow	11
8	Experiments, Observations, and Conclusions	12
8.1	Key Observations	12
8.2	Limitations	13
8.3	Conclusions	14
9	References	15

Abstract

In this report, a complete deep learning solution framework for stock market analysis and forecasting is presented. The work revolves around two main datasets – Nasdaq (U.S. stock market) and Vietnam stock market – and covers six key tasks – multiple feature price prediction, forecasting of N-th and K days ahead, detection of trading signals, creation of portfolios, risk management, implementation, and AI pipeline design.

The models used in all solutions are based on the Long Short-Term Memory (LSTM) neural networks. In order to identify trading signals, the LSTM-GradientBoosting ensemble model was utilized. In portfolio management, the forecasted returns of the LSTM models were integrated with quantitative measures of risk (such as volatility, maximum drawdown, and Sharpe ratio) to create both aggressive and conservative portfolios.

1 Introduction

Financial markets create huge volumes of sequential time-series data. Classical statistical models like ARIMA or exponential smoothing have been employed in forecasting, but they cannot detect non-linear dependencies and long-term temporal correlations in stock prices. Deep Learning and particularly LSTM networks have proven to be very powerful in modeling these types of patterns.

In this work, we implement Deep Learning on two different stock market datasets – Nasdaq Composite (the US stock market) and the Vietnam market, which consists of historical prices, dividend record, financial metrics, and industrial analysis. The goals are:

- Building multi-feature LSTM models for predicting the stock price one day, N days ahead, and consecutively for K days.
- Implementing binary classifiers for identifying buy and sell positions for stocks in the Vietnam market.
- Building and backtesting risk-managed portfolios of stocks.
- Deploying trained models as API services and designing an automated MLOps pipeline.

2 Datasets and Preprocessing

2.1 Nasdaq Dataset

The dataset from Nasdaq is comprised of separate CSV files for each individual stock, with daily Open-High-Low-Close-Volume information as well as adjusted close values.

For the purposes of this project, Apple Inc., represented by the ticker symbol AAPL, serves as the main example, using only 2018-present data.

2.2 Vietnam Dataset

The Vietnam dataset is more detailed, consisting of:

- `stock-historical-data/`: OHLCV per ticker per day
- `dividend-history/`: Dividend payment history
- `financial-ratio/`: Financial ratios (P/E ratio, ROE, Debt/Equity ratio, etc.)
- `industry-analysis/`: Industry and sector information
- `companies.csv` and `ticker-overview.csv`: Information about all listed companies

Tasks 2 and 3 are based on one company only, the FPT Corporation (ticker FPT), filtered to include 2018 onward data. Tasks 4 and 5 include eight tickers: FPT, HPG, MSN, VNM, MWG, TCB, VHM, and VIC.

2.3 Preprocessing Pipeline

- **Chronological ordering.** All data is sorted by date ascending before any processing. No shuffling is applied at any stage.
- **Company filtering.** Only companies with sufficient historical data (minimum 120 trading days post-2018) are included. Tickers with excessive missing values are excluded.
- **Train / Validation / Test split.** A strict chronological 70% / 15% / 15% split is applied. The scaler is fit exclusively on training data, then applied to validation and test sets to prevent data leakage.
- **Feature scaling.** MinMaxScaler normalises each feature to $[0, 1]$. Inverse transformation is applied before computing error metrics.
- **Sequence construction.** A sliding window of 60 trading days (~ 3 months) is used to create input sequences. Each sequence maps to the target value(s) at the forecast horizon.
- **Time-Series Cross-Validation.** TimeSeriesSplit with 5 folds (sklearn) is used on the training+validation set. This respects temporal ordering – no future data leaks into past folds.

Split	Proportion	Purpose
Training	70%	Model parameter learning
Validation	15%	Hyperparameter tuning, early stopping
Test	15%	Final unbiased evaluation

Table 1: Dataset Split Configuration

3 Task 1 — Nasdaq Stock Price Prediction

3.1 Model Architecture

All Task 1 subtasks share a common LSTM architecture:

1. Input: 60-timestep window $\times$ 5 features (Open, High, Low, Close, Volume)
2. LSTM layer 1: 128 hidden units, dropout 0.2
3. LSTM layer 2: 128 hidden units, dropout 0.2
4. Fully connected output layer: $128 \rightarrow 1$ (or $\rightarrow k$ for Task 1.3)
5. Optimiser: Adam (lr = 0.0005, weight decay = $1e^{-4}$)
6. Loss: Mean Squared Error (MSE)
7. Early stopping: patience = 15 epochs on validation loss
8. Learning rate scheduler: ReduceLROnPlateau (factor 0.5, patience 5)

3.2 Task 1.1 — Multi-Feature Extension

The baseline demo used only the Open price as input. Task 1.1 extends this to five features: Open, High, Low, Close, and Volume. The rationale is that daily price range (High – Low) encodes intraday volatility, and Volume reflects market participation – both carry predictive information beyond the opening price alone.

The model targets the next-day Close price. After training with early stopping on the validation set, the following metrics are observed on the test set:

Metric	Train	Validation	Test
MAE (USD)	\$2.81	\$4.00	\$4.62
RMSE (USD)	\$3.50	\$4.97	\$5.60
MAPE (%)	4.29%	2.47%	3.11%

Table 2: Task 1.1 Performance Metrics

The gap between training and test metrics is small, indicating the model generalises well without severe overfitting. The 5-fold Time-Series CV confirms stable performance across historical windows.

3.3 Task 1.2 — N-th Day Forecast

Task 1.2 modifies the sequence construction to target the price K days ahead instead of 1 day ahead. Three configurations are evaluated: $K = 1$, $K = 3$, and $K = 7$. A separate model is trained for each K value.

As expected, prediction error increases with forecast horizon. The MAPE for $K = 7$ is higher than for $K = 1$, reflecting greater uncertainty over longer horizons. This is consistent with the efficient market hypothesis: the further ahead one predicts, the more stochastic the price process becomes.

K (day ahead)	MAE (USD)	RMSE (USD)	MAPE (%)
1	[fill]	[fill]	[fill]
3	[fill]	[fill]	[fill]
7	[fill]	[fill]	[fill]

Table 3: Task 1.2 Forecasting Results

3.4 Task 1.3 — K Consecutive Days Forecast

Task 1.3 predicts K consecutive days simultaneously using a multi-output LSTM. The output layer is modified to produce K values (one per forecast day) rather than a single scalar. The training target is a vector of shape (N, K) where each row contains K future Close prices.

Two models are trained: $K = 3$ and $K = 7$. The average MAE across all forecast days is reported. Day-level plots show that prediction accuracy degrades gradually as the forecast day increases within the horizon, which is the expected behaviour for direct multi-step forecasting.

4 Task 2 — Vietnam Stock Price Prediction

4.1 Data Specifics

The Vietnam market varies structurally from the Nasdaq market as follows: prices are quoted in VND (Vietnamese Dong); trades take place at the Ho Chi Minh Stock Exchange (HOSE); and different trading times apply. The ticker for FPT Corporation is chosen as it has been trading for some time now (after 2018) and has sufficient liquidity.

The same WINDOW = 60, 70/15/15 ratio, and LSTM model as in Task 1 are used again but adjusted accordingly; the scaling is based on Vietnamese stock prices.

4.2 Task 2.1 — Multi-Feature Extension

Identical in structure to Task 1.1 but applied to FPT data. Five features (Open, High, Low, Close, Volume) are used. The model targets the next-day Close price in VND.

Metric	Train	Validation	Test
MAE (VND)	[fill]	[fill]	[fill]
RMSE (VND)	[fill]	[fill]	[fill]
MAPE (%)	[fill]	[fill]	[fill]

Table 4: Task 2.1 Performance Metrics

4.3 Task 2.2 — N-th Day Forecast

The same approach as Task 1.2 is applied to FPT data. Three horizons ($K = 1, 3, 7$) are evaluated. A separate LSTM is trained per K .

K (day ahead)	MAE (VND)	RMSE (VND)	MAPE (%)
1	1,715	2,217	2.21%
3	1,858	2,483	2.39%
7	3,180	4,014	4.09%

Table 5: Task 2.2 Forecasting Results

4.4 Task 2.3 — K Consecutive Days Forecast

Multi-output LSTM predicting $K = 3$ and $K = 7$ consecutive future days for FPT. Results follow the same pattern as Task 1.3: accuracy decreases with forecast day index within the horizon.

5 Task 3 — Trading Signal Identification

5.1 Problem Formulation

Trading signal identification is framed as a binary classification problem. Two separate models are trained:

1. Buy Signal: label = 1 if the stock price increases by more than 2% over the next 5 trading days.

2. Sell Signal: label = 1 if the stock price decreases by more than 2% over the next 5 trading days.

The threshold of 2% and horizon of 5 days are chosen to filter out noise while capturing meaningful directional moves. These hyperparameters can be adjusted depending on the trader's risk tolerance.

5.2 Feature Engineering

Raw price data alone is insufficient for classification – technical indicators derived from price and volume provide richer, non-redundant signals. The following indicators are computed:

Feature	Formula / Meaning
SMA_10, SMA_20	Simple Moving Average over 10 and 20 days
EMA_12, EMA_26	Exponential Moving Average (MACD components)
MACD	EMA_12 - EMA_26 (trend momentum)
MACD_signal	9-day EMA of MACD
MACD_hist	MACD - MACD_signal (histogram)
RSI	Relative Strength Index over 14 days
BB_width	Bollinger Band width ($4 * std_{20}/sma_{20}$)
return_1d, return_5d	1-day and 5-day percentage returns
Close_vs_SMA10/20	Close price relative to moving averages
Volume_change	Daily volume percentage change

Table 6: Technical Indicators Used for Classification

5.3 Model Architecture: LSTM + Gradient Boosting Hybrid

A two-step process is followed. First, an LSTM model generates temporal features based on the input of 60 days. Next, a Gradient Boosting model uses both generated features and the latest raw values to output its predictions.

Stage 1 — LSTM Model:

Input dimension 9 (technical features count), hidden dimension 64, 2 layers, 0.3 dropout rate, BatchNorm activation. Loss function: BCEWithLogitsLoss with `pos_weight` to account for class imbalances. Optimiser: Adam with CosineAnnealingLR. Validation-based early stopping using F1 metric.

Stage 2 — Gradient Boosting:

Concatenation of 64 LSTM hidden layer values and the last day of raw feature values (9 dimensional). Train GradientBoostingClassifier with maximum `n_estimators` of 500, `max_depth=3`, and `subsample=0.7`. Determine optimal `n_estimators` through staged_predict_proba iterations on validation set.

Rationale: While LSTM works well with detecting temporal relationships, it may struggle with defining decision boundaries. Gradient Boosting introduces nonlinear interactions between generated features and provides a structured feature space.

5.4 Results

Signal	Accuracy	F1 Score	Precision	Recall	Threshold
Buy	0.617	0.294	0.35	0.26	0.50
Sell	0.615	0.297	0.24	0.38	0.50

Table 7: Trading Signal Classification Results

Class imbalance is addressed via `pos_weight` in `BCEWithLogitsLoss` and `min_samples_leaf=20` in GBC. The F1 score is the primary metric given the imbalanced nature of trading signals.

6 Task 4 — Portfolio Composition and Risk Management

6.1 Universe Selection

Eight Vietnamese blue-chip stocks are selected from diverse sectors to enable diversification:

Ticker	Company	Sector
FPT	FPT Corporation	Technology
HPG	Hoa Phat Group	Steel / Manufacturing
MSN	Masan Group	Consumer Goods
VNM	Vinamilk	Food & Beverage
MWG	Mobile World	Retail
TCB	Techcombank	Banking / Finance
VHM	Vinhomes	Real Estate
VIC	Vingroup	Conglomerate

Table 8: Selected Vietnamese Blue-Chip Stocks

All tickers share a common date range, and prices are aligned on common trading days. A daily returns matrix is computed as the basis for risk analysis.

6.2 Task 4.1 — Profitable Stock Selection

An LSTM model (5 features, 64 hidden units, 2 layers) is trained for each ticker using the last 85% of data. The model then performs autoregressive forecasting for the next 20

trading days (~ 1 month): at each step, the predicted Close is fed back into the window as the new observation.

Projected return is defined as:

$$\text{Projected Return} = \frac{\text{Forecast Price} - \text{Current Price}}{\text{Current Price}} \times 100\%$$

Tickers are ranked by projected return. Those with a positive projected return are considered candidates for inclusion in the portfolio.

6.3 Task 4.2 — Risk Management

Three risk dimensions are computed for each ticker using the most recent 252 trading days (~ 1 year):

1. Annualized Volatility: standard deviation of daily returns $\times \sqrt{252}$
2. Maximum Drawdown: the largest peak-to-trough decline in cumulative returns

3. Sharpe Ratio:

$$\frac{\text{mean daily return} - \text{risk-free rate}}{\text{std}} \times \sqrt{252}$$

assuming a 5% annual risk-free rate.

4. Sortino Ratio: similar to Sharpe but penalizing only downside deviation.

A composite risk score is computed as a weighted combination:

$$\text{Risk Score} = 0.4 \times \text{Volatility} + 0.4 \times |\text{Max Drawdown}| + 0.2 \times \max(0, -\text{Sharpe})$$

Higher risk scores indicate riskier assets.

6.4 Task 4.3 — Portfolio Composition

Aggressive portfolio (risk-taking investor): All tickers with positive projected return are included. Portfolio weights are proportional to the reward/risk score.

Conservative portfolio (prudent investor): Only tickers with positive projected return and below-median risk score are included.

Both portfolios are back tested on the out-of-sample test set (last 15% of the common date range).

Portfolio	Tickers held	Total Return (%)	Sharpe	Max Drawdown (%)
Aggressive	VIC, VHM, HPG, TCB, MWG, VNM, MSN	-26.62%	-1.33	-33.12%
Conservative	VIC, VNM, MSN	-26.84%	-1.22	-27.47%

Table 9: Portfolio Performance Comparison

7 Task 5 — Deployment and AI Engineering (Extra Credit)

7.1 Task 5.1 — Model Deployment as API

The trained LSTM model (Task 1.1 weights) is exported and served via a FastAPI REST endpoint. The service accepts a JSON payload containing 60 days of OHLCV data and returns a predicted next-day Close price.

Deployment steps:

1. Export model weights: `torch.save(model.state_dict(), 'lstm_model.pth')`
2. Save scaler parameters (`min_`, `scale_`) as a pickle file.
3. Build a FastAPI application with a `/predict` endpoint.
4. Containerise with Docker for reproducible deployment.

Example request:

```
curl -X POST http://localhost:8000/predict \
-H "Content-Type: application/json" \
-d '{"data": [...60 rows of OHLCV...]}'
```

7.2 Task 5.2 — Model as SaaS

A lightweight web interface is built using Streamlit, allowing users to upload or manually input recent stock data and receive a predicted price. The interface calls the FastAPI backend and displays the result alongside a chart of the recent price history.

7.3 Task 5.3 — AI Engineering Workflow

An automated MLOps pipeline is designed to handle the full lifecycle from raw data ingestion to prediction storage.

The Airflow DAG runs daily at market close. It triggers Airbyte to sync new price data, then runs dbt transformations, executes the prediction script, and finally writes results to the predictions table.

Component	Tool	Role
Data Ingestion	Airbyte	Pull daily stock data from market APIs into PostgreSQL
Data Transformation	dbt	Compute rolling features (SMA, RSI, MACD, volatility)
Orchestration	Apache Airflow	Schedule daily DAG
Storage	PostgreSQL	Store raw prices and predictions
Serving	FastAPI + Docker	Expose predictions via REST API
Visualisation	Streamlit	Dashboard for portfolio view and signal alerts

Table 10: MLOps Pipeline Components

8 Experiments, Observations, and Conclusions

8.1 Key Observations

1. LSTM multi-feature models invariably surpass single-feature models, establishing that other market indicators, like volume and intraday variability, hold predictive significance. Relative to LSTM models that learn from open prices alone, those incorporating High, Low, Close, and Volume yield more robust results during volatile market conditions while minimizing prediction errors on new observations. This highlights that stock prices cannot be adequately captured by a single variable since the markets are inherently multi-dimensional.
2. The accuracy of the forecast decreases progressively with the lengthening of the forecasting period. Such behavior is anticipated in financial time series forecasting due to the uncertainty that builds up over time as well as the impact that stock prices experience from various exogenous stochastic inputs like market sentiments and macroeconomic news. The performance measure MAPE for $K = 7$ forecast is found to be larger than $K = 1$.
3. The LSTM-based prediction model which was designed to predict the next day’s trading trend based on the previous day’s data is able to pick up the temporal dependencies between future days accurately. However, the performance becomes lower with the increasing time distance. This proves that despite being able to handle temporal data accurately, the LSTM algorithm is prone to accumulate errors over time.
4. It turns out that LSTM+Gradient Boosting classifier works much better compared to the LSTM classifier when dealing with financial trading signals classification. It is because LSTM can produce an accurate temporal representation of data and then Gradient Boosting can make use of it when performing classification.

5. Some of the technical indicators that are used as inputs include RSI, MACD, Bollinger Band width, and moving averages. The models using indicators as features perform better than those using just OHLCV data as inputs. Momentum-related indicators particularly assist in improving signal generation in conditions of high market movement.
6. Time-series cross-validation is essential for financial forecasting tasks. Traditional random k-fold cross-validation introduces future data leakage because observations from later time periods may appear in the training set while earlier periods are used for validation. By using TimeSeriesSplit, the evaluation process better reflects real-world forecasting scenarios and produces more reliable generalisation estimates.
7. Portfolio risk management significantly affects investment stability. Although the aggressive portfolio includes more stocks with higher projected returns, it experiences larger maximum drawdown and lower Sharpe ratio compared with the conservative portfolio. This observation is consistent with modern portfolio theory, where higher expected returns are usually associated with higher levels of risk exposure.
8. The Vietnam stock market exhibits different characteristics from the Nasdaq market, including lower liquidity, higher volatility in certain periods, and shorter historical records for some tickers. These factors increase forecasting difficulty and may limit the effectiveness of deep learning models compared with more mature markets such as the United States stock market.

8.2 Limitations

1. The Vietnam market dataset is relatively smaller and less consistent compared with the Nasdaq dataset. Some companies have insufficient historical records or missing trading periods, which reduces the effective amount of training data available for deep learning models. Limited data diversity may also reduce the generalisation capability of the trained models.
2. Multi-step autoregressive forecasting leads to the accumulation of forecast errors over time since each successive forecast will use a previous forecast as its predictor. Even small errors committed at the start of the process may amplify as the forecasting horizon stretches farther into the future.
3. All the models are solely based on the historical data related to stock prices together with technical indicators, while there is no incorporation of external information in any of the models, including the effect of macroeconomic factors, financial news, market sentiment, and political events. But in reality, such external information may become an important determinant in the movement of stock prices.

4. The portfolio backtesting framework is simplified and does not account for realistic trading constraints such as transaction costs, taxes, slippage, liquidity limitations, or order execution delays. Consequently, the reported portfolio returns and Sharpe ratios may be more optimistic than real-world trading performance.
5. Hyperparameter tuning is limited due to computational constraints. Although early stopping and validation-based optimisation are applied, more extensive hyperparameter search methods such as Bayesian optimisation or large-scale grid search may further improve model performance.
6. The current implementation focuses mainly on LSTM architectures and does not evaluate more advanced sequential models such as Transformers, Temporal Fusion Transformers (TFT), or attention-based architectures. These modern approaches may better capture long-range dependencies and complex temporal relationships in financial time-series data.
7. Trading signal classification suffers from class imbalance because strong upward or downward movements occur less frequently than neutral market behaviour. Although techniques such as weighted loss functions and minimum leaf constraints are applied, the resulting F1 scores remain relatively modest, indicating that trading signal prediction remains a challenging problem.
8. The released web application is trained only on AAPL stock data from a limited price range. Since the scaler and model parameters are fitted specifically to this distribution, applying the model directly to stocks with significantly different price scales or market structures may produce inaccurate predictions. A production-level system would require ticker-specific scaling or retraining pipelines.

8.3 Conclusions

The experiment shows that deep learning models based on LSTM are useful tools for stock price forecasting and trading signal detection in both the American and Vietnamese stock markets.

The hierarchical methodological framework from stock price forecasting to portfolio formation represents a practical quantitative investment approach. The suggested MLOps pipeline can automate and scale the process.

The future directions may involve transformer-based architectures such as the Temporal Fusion Transformer, sentiment analysis integration, and advanced portfolio optimisation methods like mean-variance optimisation or Black-Litterman.

9 References

1. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
2. Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.
3. Markowitz, H. (1952). Portfolio selection. *The Journal of Finance*, 7(1), 77–91.
4. Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *NeurIPS*.
5. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825–2830.
6. Apache Airflow Documentation. <https://airflow.apache.org/docs/>
7. dbt Documentation. <https://docs.getdbt.com/>
8. Airbyte Documentation. <https://docs.airbyte.com/>